Lovely Tech POS System

Database / Technical Blueprint — v2.0

FINALIZED IMPLEMENTATION BASELINE

Consolidated developer hand-over specification. Locked business rules are stated as implementation requirements; historical source material was reviewed for terminology and repair-form fields.

01 Architecture

• Use a proper local DB server/client architecture over LAN. Do not share a SQLite/database file directly over the network.

• One laptop hosts the local DB server during normal operation; other laptops are clients. Internet is not required for core LAN operation.

• Google Drive is backup/recovery only. Architecture is branch-aware from the beginning.

02 Identity / Numbering

• Every entity uses an internal UUID primary key.

• Human-facing sequential numbers are separate: INV, REP, RET, EXC, REC, MWR and configurable future prefixes.

• Number allocation is atomic; gaps may remain after void/cancel; numbers are never reused. Item codes follow the same non-reuse rule.

03 Data Domains

• Master: branches, users, roles, permissions, categories, item_types, items, item_barcodes, variants, warranty_defaults, suppliers, expense_categories, targets.

• Transactions: sales, sale_lines, payments, returns, return_lines, exchanges, purchases, purchase_lines, repairs, repair_parts, repair_payments, product_warranties, warranty_claims, repair_warranties, credit_settlements, expenses, other_income, day_sessions, day_ends, reload_entries, other_cash_in.

• Stock/FIFO: purchase_lots, stock_movements, fifo_allocations, stock_adjustments, damage_loss_records, warranty_holdings, serialized_units.

• Security/history: approvals, audit_log, login_events, document_events, backup_points, restore_events, idempotency_keys, record_versions.

04 Sales / Payments

• sales: id UUID PK; branch_id FK; invoice_no UNIQUE; customer_id nullable; sale_datetime; cashier_id; document_type SYSTEM_PRINTED/MANUAL/NO_PRINT; manual_invoice_ref nullable; subtotal; discount_total; grand_total; status; void_reason; timestamps.

• sale_lines store item, quantity, actual unit price, discount, final total and serialized-unit references where required.

• payments are separate rows so split payment is represented without overwriting one method.

• Manual reference is required when document_type=MANUAL and is indexed for recovery search.

05 Item / Purchase / FIFO

• purchase_lots preserve original quantity, date and cost lineage.

• stock_movements are append-oriented events with source type/id, item, quantity delta, datetime, branch and cost lineage.

• fifo_allocations link each stock-consuming movement to one or more purchase lots and allocated quantity/cost.

• Negative-stock reconciliation is deterministic and transaction-safe. Later purchase lots keep their full original quantity/cost.

• Purchase cost/date/quantity correction invalidates and recomputes affected FIFO allocations and dependent profit projections while preserving audit.

• Serialized items have unique active IMEI/serial constraints.

06 Returns / Exchange

• Returns are independent transaction records. Good return creates positive stock movement with original lineage; damaged return creates Damage/Loss.

• Exchange is a wrapper linking the old return and the replacement sale; both remain independently auditable.

• Return charge is a distinct income component.

07 Repair

• Repair header stores REP number, status, device/job/customer fields, estimate, final/labour amounts, dates, warranty and authorization metadata.

• repair_parts link item, quantity, charge/value where applicable, stock movement and FIFO allocations.

• repair_payments are payment events; outstanding is derived from payment history.

• Cancellation reverses good/unused part stock and records Damage/Loss for damaged parts. Payment history is never erased.

08 Warranty

• product_warranties link to sale_line/serialized_unit or exception manual registration; store start, expiry, type, source and status.

• manual_warranty_documents are exception-only.

• warranty_document_events store ORIGINAL/REPRINT/DUPLICATE/RECOVERY.

• warranty_claims store Admin decision, supplier process, holding quantity and replacement outcome.

• Supplier replacement is WARRANTY_REPLACEMENT, not PURCHASE.

09 Approval / Permissions

• approval_requests store action type, source transaction, requester, payload/reference, status PENDING/APPROVED/REJECTED, reviewer, timestamps and reject reason.

• Unique approval/action IDs prevent duplicate execution. Reject reason is mandatory.

• Pending actions do not enter final financial projections until approved.

10 Concurrency / Atomicity

• Critical effects use DB transactions with commit/rollback.

• Editable records use version fields; stale Cashier edits cannot overwrite newer Admin changes.

• Use row locking where required for stock/FIFO and document-number allocation.

• Idempotency keys protect retry after timeout/crash.

• Never allow two clients to commit the same invoice number, IMEI/serial, approval action or stock mutation.

11 Audit Model

• audit_log should include id, branch_id, actor_user_id, action_type, entity_type, entity_id, timestamp, reason, before_json, after_json, source_transaction_id and security/context metadata.

• Audit is append-only and protected from normal update/delete.

• Reprints, security events, permission changes and backup/restore are audited even without stock change.

12 Cash / Financial Projections

• Cash Drawer and Cash In Hands are separate projections.

• Cash Drawer expected uses opening cash + drawer-relevant inflows – refunds/outflows + Day-End reload total under the locked rule.

• Cash In Hands includes all approved inflows/outflows including purchases and expenses.

• Profit uses transaction revenue, exact FIFO cost, repair/labour profit, approved other income, return charges and applicable losses/expenses.

• Credit/repair settlements are payment events only.

13 Backup / Restore

• Backup states: CREATED, ENCRYPTED, UPLOADED, VERIFIED, FAILED/INCOMPLETE. Only VERIFIED is restorable.

• Emergency local backup uses a separate configured storage target; it is not merely a same-folder database copy.

• Restore: Admin re-authentication → safety backup → confirmation → selected verified point → integrity checks → audit.

• Queued backups retry after connectivity returns.

• Disaster recovery must avoid split-brain databases and uncontrolled merge.

14 Constraints / Indexes

• Unique constraints: invoice number, item code per defined scope, active IMEI/serial, configured manual reference policy, approval action ID and idempotency key.

• Indexes: barcode, item search fields, invoice number, manual invoice reference, IMEI/serial, sale datetime, repair number, warranty status/expiry, approval status, pending purchase cost, stock movement item/date and FIFO lot.

• Foreign keys protect historical references. Important records are voided/corrected, not hard-deleted.

• Money uses fixed-point DECIMAL, never floating point.

15 State Machines

• Sale: DRAFT → COMPLETED → optional PARTIAL_RETURN/VOID through controlled action.

• Repair: RECEIVED → CHECKING → WAITING_FOR_CUSTOMER / WAITING_FOR_PART → REPAIRING → READY → DELIVERED/RETURNED_TO_CUSTOMER; CANCELLED terminal.

• Warranty: CREATED/PENDING_ADMIN → SUPPLIER_CLAIM or DAMAGE_LOSS → accepted replacement/rejected/customer-keeps-item → CLOSED.

• Approval: PENDING → APPROVED or REJECTED.

• Backup: CREATED → ENCRYPTED → UPLOADED → VERIFIED; failed branches never become verified.

• Allowed transitions must be enforced server-side.

16 Recalculation / Acceptance

• Recalculate affected FIFO and dependent profit/stock projections after purchase cost/date/quantity correction, stock correction, valid return reversal/correction, damage/loss correction or any cost-lineage change.

• Do not rewrite historical transactions merely to fix reports; store the correction and recompute derived values.

• Acceptance tests must cover power loss, printer failure, double-click, timeout, concurrent edit, permission denial, negative stock then purchase, multiple lots, return/resale, repair consumption, damage/loss, cost/date correction, credit settlement, repair settlement, backup interruption and restore interruption.

• Security acceptance: Cashier must be unable to retrieve hidden cost/profit through UI or API; Admin-only actions must fail server-side for unauthorized users.